

INGEN DYNAMICS INC.

Origami AI · Physical Intelligence Platform (PIC 2.0)

INTERNSHIP PROGRAM PLAN

SDE Intern · 8-Week Remote Program · Mon–Sat · v1.0

Intern	Calvin Wang
Position	SDE Intern — Robot Perception Services & Backend Platform Track
Engagement	Internship (Fully Remote, 8 weeks + final readout)
Education	Purdue University, West Lafayette, IN — B.S. Computer Engineering, expected May 2028 · GPA 3.94/4.0 · Concentration: Artificial Intelligence and Machine Learning · Dean's List and Semester Honors
Key Experience	Clash Royale AI Agent — Technical Cofounder (Aug 2025–Present): YOLOv10 fine-tuning on custom data, real-time perception pipeline, structured game-state encoding, self-play training loop · Stock Research Website — Full Stack Developer (Jun–Jul 2025): FastAPI backend, HTML/CSS/JS frontend, third-party market API integration, OpenAI model integration with prompt engineering · ML Model for Stock Analysis — ML Engineer (Jul–Aug 2025): yfinance data pipeline, feature engineering, hyperparameter tuning, backtesting against a baseline · 16 Handles — Manager (2022–2024): scheduling, inventory, operations
Technical Stack	Python · Java · JavaScript · C · C# · MATLAB · Swift · PyTorch · FastAPI · OpenAI API · Git / GitHub · Microsoft Excel
Week Structure	Monday to Saturday (6-day weeks) · Final Readout: Monday November 9
Program Dates	September 14, 2026 (Monday) – November 7, 2026 (Saturday) · Final Readout November 9
Mid-Point Review	Saturday, October 10, 2026 (end of Week 4) · 09:00 EDT / 18:30 IST
Final Readout	Monday, November 9, 2026 · 09:00 EST / 19:30 IST
Time Zone	Eastern Time (EDT, UTC-4 through Oct 31; EST, UTC-5 from Nov 1) · Supervisor: IST (UTC+5:30) · Offset: 9.5 hours, becoming 10.5 hours after US DST ends on November 1
Sync Time	Saturday syncs: 09:00 ET / 18:30 IST (19:30 IST after Nov 1) · Interview availability: any reasonable hours Eastern Time
Contact	wcalvin2006@gmail.com · wang6255@purdue.edu · linkedin.com/in/calvinwang16 · 908-666-2821
Supervisor	Iqbal Patel — HR & Admin Manager, InGen Dynamics Inc.
Primary Deliverable	ingen_perception_service_v1.0 GitHub repository: a containerised FastAPI perception and telemetry service for InGen robot platforms, with a YOLO-based detection worker, a state-encoding layer, a persistence and query API, and a test and benchmark suite
Document Version	v1.0 · September 2026 · Confidential — Internal Use

Perception Service API · Robot State Encoder · Telemetry Store & Query Layer · Detection Worker & Benchmark
Harness · ingen_perception_service_v1.0

1. Program Overview

1.1 Purpose

This plan structures an 8-week, fully-remote SDE internship for Calvin Wang at InGen Dynamics along the Robot Perception Services and Backend Platform track. The programme is divided into four two-week phases: (A) Platform landscape orientation and service foundation — understanding each InGen robot platform from a backend engineer's perspective, making the service architecture and data-contract decisions, and standing up the FastAPI application skeleton with health checks, configuration, logging, and CI; (B) Perception worker and state encoding — wrapping a YOLO detection model behind a stable inference interface, defining and implementing the robot state encoding layer, and adding the frame ingestion path, capped by the mid-point review at the end of Week 4; (C) Persistence, query API, and event pipeline — a telemetry store with a documented schema, a query and filtering API for detections and robot state, an event/alert rule engine, and an operator-facing dashboard endpoint; and (D) Reliability, performance, and release — load testing, latency profiling, containerisation, observability, and the final `ingen_perception_service_v1.0` repository with a 10-slide technical capstone deck delivered at the final readout on Monday November 9.

The primary deliverable of this programme is a GitHub-ready backend service — `ingen_perception_service_v1.0` — that InGen's robotics and AI teams could run as the perception and telemetry backend for Aido Rover and Fari. Every engineering decision across the 8 weeks is evaluated against that purpose: is this API contract stable enough that a robot client and a dashboard client can both build against it without coordination? Does this state encoding capture enough of the world for a decision policy to act on it? Does this service still meet its latency budget when ten robots stream frames at once, and can an on-call engineer tell from the logs and metrics why it did not?

1.2 Why This Programme Fits Calvin's Profile

- **Clash Royale AI Agent — direct technical precursor:** The Clash Royale AI Agent project (Aug 2025–Present) is the closest thing in Calvin's background to what this programme builds. He already assembled the same three-stage pipeline InGen needs: a computer vision stage (YOLOv10 fine-tuned on custom generated data to detect board elements from live gameplay footage), a state-modelling stage (transforming raw vision outputs into a structured state encoding), and a decision stage consuming that state. The perception service in this programme is that pipeline turned into a production backend — the detector becomes a worker behind a versioned inference interface, the state encoding becomes a documented schema with a JSON contract, and the decision consumer becomes an external client served over HTTP. The Clash Royale work is not background context; it is the architecture this programme hardens.
- **Stock Research Website — full-stack service delivery:** The Stock Research Website (Jun–Jul 2025) is the FastAPI precedent. Calvin designed and built a full-stack application with a Python FastAPI backend, a browser frontend, third-party API integration, and an LLM integration with prompt engineering — end to end, shipped, and used. That is exactly the shape of Weeks 1–3 and Week 6 here: define the API, implement it in FastAPI, integrate an upstream data source that you do not control, and put a usable interface in front of it. He is not learning FastAPI in Week 1; he is being held to a production standard in it.
- **ML Model for Stock Analysis — data pipeline and honest evaluation:** The stock analysis model (Jul–Aug 2025) required collecting, cleaning, and processing financial data through a Python pipeline, then improving results through feature engineering and hyperparameter tuning, and — importantly — backtesting against a random baseline rather than reporting only favourable results. That evaluation discipline is what the Week 7 benchmark harness demands: a measured baseline, a measured post-optimisation result, and an honest statement of what did not improve. An engineer who has already compared a model against a baseline understands why the benchmark comes before the optimisation.
- **Computer Engineering at Purdue — systems fundamentals under the API:** Purdue Computer Engineering with an AI/ML concentration, at a 3.94 GPA, with Data Structures, Linear Algebra, Discrete Math, and Advanced Programming in C completed. The C coursework matters more than it looks: understanding memory layout, buffers, and the cost of a copy is what separates a perception service that handles image payloads efficiently from one that quietly serialises a 4 MB frame four times per request. The data structures grounding is what makes the Week 5 query and indexing design a considered decision rather than a default.
- **Multi-language breadth — Java, C#, Swift, JavaScript:** Calvin works across Java, C#, C, JavaScript, Swift, and MATLAB in addition to Python. A perception backend at a robotics company is never consumed only by Python: the robot client

may be C++ or C#, the dashboard is JavaScript, and a mobile operator app is Swift or Kotlin. The Week 3 API contract work explicitly requires designing an interface that a non-Python client can consume cleanly, and Calvin has the cross-language experience to judge whether the contract is actually language-neutral.

- **16 Handles management — operational ownership:** Two years managing scheduling, inventory tracking, and store operations (Jun 2022–Aug 2024). It is not a software credential, but it is the reason the async operating model in Section 5 is a reasonable expectation rather than a hope: someone who has run shift scheduling and inventory has already learned to communicate status before being asked, flag a shortage early, and keep a system running when the manager is not in the room. Across a 9.5-hour time offset, that is the operative skill.

1.3 Background to Programme Bridge

Calvin's Experience / Project	Maps To (Programme Task)	Phase & Week
Clash Royale AI Agent: YOLOv10 fine-tuning; real-time detection from live footage	Week 3 detection worker: wrapping a YOLO model behind a versioned inference interface with warm-up, batching, and a documented latency budget	Phase B — Week 3
Clash Royale AI Agent: transforming vision outputs into a structured state encoding for decision making	Week 4 robot state encoder: the RobotState schema, entity list, confidence handling, and JSON contract consumed by downstream clients	Phase B — Week 4
Clash Royale AI Agent: self-play loop and repeated evaluation against specialised agents	Week 7 benchmark harness: reproducible, scripted load and accuracy runs producing comparable numbers across builds	Phase D — Week 7
Stock Research Website: FastAPI backend, endpoint design, request/response handling	Weeks 1–2 service foundation: FastAPI application skeleton, routing, Pydantic models, config, health and readiness endpoints	Phase A — Wks 1–2
Stock Research Website: third-party market API integration; handling an upstream you do not control	Week 5 ingestion and Week 6 event pipeline: timeouts, retries, backpressure, and graceful degradation when an upstream or a worker is unavailable	Phase C — Wks 5–6
Stock Research Website: HTML/CSS/JS frontend built on his own API	Week 6 operator dashboard endpoint and minimal browser view for live detections and robot state	Phase C — Week 6
ML Stock Model: yfinance pipeline — collection, cleaning, processing	Week 5 telemetry store: ingestion path, schema design, retention policy, and data-quality checks on inbound frames and detections	Phase C — Week 5
ML Stock Model: backtesting against a random baseline; honest reporting of directional accuracy	Week 7 performance report: measured before/after latency, throughput, and error rates, with unresolved bottlenecks named	Phase D — Week 7
Purdue: Data Structures; Advanced Programming in C	Week 5 query layer and indexing decisions; Week 7 payload handling and memory profiling on the image path	Phase C–D — Wks 5, 7
16 Handles: scheduling, inventory, operations ownership	Section 5 async operating model: daily notes by 09:00 ET, blockers with stated default assumptions, Saturday syncs prepared in advance	All phases

1.4 Operating Constraints

- All models, datasets, and sample media are open-licensed or self-generated: Ultralytics YOLO pretrained weights under their stated licence, COCO or other public sample imagery, and synthetic or self-recorded test frames. No proprietary InGen data, no customer footage, and no images of identifiable people not consented to.
- `ingen_perception_service_v1.0` is organised as a GitHub-safe repository: no InGen confidential content, no credentials or API keys in code or history, no proprietary model weights. Configuration is supplied by environment variables with a committed `.env.example`. All non-obvious code is commented.
- The service targets a documented latency budget: p95 end-to-end inference latency under 250 ms per 640×640 frame on CPU-only commodity hardware, and under 100 ms where a GPU is available, measured in Week 3 and re-measured in Week 7. If the target cannot be met, the gap is reported with a technical explanation rather than quietly relaxed.

- The API contract is versioned from Week 2 onward (/v1/ prefix). Once an endpoint is published in a weekly deliverable, breaking changes require a documented migration note. This is the discipline that makes the service usable by a client team.
- Every technical decision is documented: why this storage engine, why this queue model, why this state schema, what the timeout and retry values are, and what happens when the detector is unavailable.

2. Programme Arc — Four Two-Week Phases

Wks	Dates	Phase	Primary Engineering Lens	Primary Output
1–2	Sep 14 – Sep 26	A: Landscape & Service Foundation	Platform requirements analysis; API contract design; FastAPI application structure; config, logging, testing, and CI baseline	Platform brief + API design doc + running FastAPI service skeleton with CI green
3–4	Sep 28 – Oct 10	B: Perception Worker & State Encoding	Model serving; inference interface design; latency measurement; structured state modelling; schema versioning	Detection worker v0.1 + RobotState encoder + frame ingestion endpoint + Mid-Point
5–6	Oct 12 – Oct 24	C: Persistence, Query & Events	Data modelling and indexing; query API design; asynchronous event processing; rule evaluation; operator UI	Telemetry store + query API + event/alert engine + operator dashboard endpoint
7–8	Oct 26 – Nov 7	D: Reliability, Performance & Release	Load testing; profiling; containerisation; observability; documentation and release engineering	Benchmark harness + performance report + Docker deployment + ingen_perception_service_v1.0 release + 10-slide capstone deck

2.1 InGen Platform Service Anchors

Platform	Weeks	SDE / Backend Relevance
Aido Rover (outdoor patrol)	1–3, 5–8	PRIMARY SERVICE TARGET. Phases A through D are centred on the perception and telemetry backend for an outdoor patrol robot. The core backend problem: a patrol robot streams camera frames continuously from a moving platform over an unreliable network, needs detections back fast enough to act on them, and generates a continuous telemetry record that an operator must be able to query after the fact ('what did unit 3 see near the loading dock between 02:00 and 03:00?'). That is a throughput problem, a latency problem, a schema problem, and a query problem at once — which is why it is the primary anchor. Calvin's Clash Royale pipeline solved the same real-time detection-to-state loop at a smaller scale.
Fari (indoor companion)	4, 6, 8	SECONDARY SERVICE TARGET — Weeks 4 and 6. Fari's backend requirements differ from Aido Rover's in ways that stress the API design: close-range recognition rather than distance detection, event-driven interactions rather than continuous patrol telemetry, and stricter privacy constraints on what may be stored from a domestic or eldercare interior. Fari is the test of whether the Week-2 API contract and the Week-4 state schema generalise across platforms, or whether they were quietly specialised to Aido Rover.
Sentinel Prime AI (security)	1, 6, 8	EVENT PIPELINE CONTEXT — Week 1 platform brief, Week 6 alert engine, Week 8 capstone. A security platform is the natural consumer of the Week-6 event and alert rule engine: detection of a defined object class in a defined zone during a defined time window generates an alert with a severity and an audit trail. Noted in the Week-1 brief; the alert engine is designed with this consumer in mind; recommended as a next step in the capstone deck.
Aido Humanoid (bipedal research)	1, 8	ADVANCED CONTEXT — Week 1 platform brief. The humanoid research platform would demand the tightest control-loop latency and the richest multi-sensor state fusion of any InGen platform, and would likely require a native transport rather than HTTP. Noted in the Week-1 brief as the case that would stress the service architecture hardest; Week 8 capstone next-steps recommendation.

Platform	Weeks	SDE / Backend Relevance
Senpai (educational robot)	1, 8	LANDSCAPE CONTEXT — Week 1 platform brief. Senpai's backend would emphasise session state, per-student interaction history, and content delivery over high-rate perception. Noted in the brief as the platform whose requirements least resemble the primary service; capstone next-steps.

3. Week-by-Week Plan

Eight weeks of backend software engineering, from service foundation through perception serving, persistence and events, reliability work, and GitHub release. Primary service target: Aido Rover (outdoor patrol). Secondary: Fari (indoor companion). Primary stack: Python 3.11, FastAPI, Pydantic v2, PyTorch/Ultralytics, SQLite or PostgreSQL (decision documented in Week 1), Docker, pytest, GitHub Actions. Final readout: Monday November 9.

PHASE A · Platform Landscape & Service Foundation · Weeks 1–2 · Sep 14 – Sep 26

Week 1 · Sep 14–19, 2026 (Mon–Sat) · InGen Backend Landscape, Architecture Decision & Service Skeleton	
Platform Anchor	All five InGen platforms — backend and data requirements survey · Aido Rover (primary design begins)
Objective	Build a working understanding of each InGen platform's backend service requirements from an SDE's perspective. Make and document the core architecture decisions (storage engine, sync vs async inference, transport). Stand up the FastAPI service skeleton to production standard with CI green on day six.
Tasks	InGen backend platform brief (W01_Backend_Platform_Brief.docx): For each of InGen's five platforms, write an SDE's analysis covering: (a) Data profile — what does this robot produce, at what rate, and in what size? (frame rate, image resolution, expected payload size per second, burst behaviour); (b) Latency requirement — does this platform need a synchronous inference response in the control loop, or is asynchronous processing with an eventual result acceptable? State the tolerable p95 latency and why; (c) Storage and retention — what must be persisted (raw frames, detections only, aggregated state), for how long, and what are the privacy constraints on that data? Fari in a private residence and Aido Rover in a commercial car park are different answers; (d) Client shape — who consumes this service? (on-robot client, operator dashboard, mobile app, another internal service) and what does that imply for the transport and the contract; (e) Primary engineering challenge — the hardest backend problem for this platform. This brief is the technical foundation for every subsequent design decision. Architecture decision record (include in the brief, ADR format): Make and justify three decisions in two paragraphs each. (1) Storage engine: SQLite versus PostgreSQL for the telemetry store. Consider the write rate implied by the Week-1 data profile, the query patterns in Week 5, concurrent-writer behaviour, and the cost of running the service on a single machine for a demo versus a real deployment. (2) Inference execution model: in-process synchronous inference versus a separate worker process with a queue. Consider the latency budget, the GIL, model memory, and what happens to the API when the model is loading or has crashed. (3) Transport: REST with multipart image upload versus WebSocket streaming for the frame path. State each choice and commit to it — changing the storage engine or execution model in Week 6 is a significant and avoidable cost. Service skeleton (W01_Service_Skeleton — GitHub repo): Create the repository and the FastAPI application skeleton. Required: (a) project layout — app/ (api/, core/, models/, services/, workers/), tests/, docs/, scripts/; (b) dependency and environment management with a pinned lockfile (uv, Poetry, or pip-tools — state the choice); (c) configuration via environment variables using pydantic-settings, with a committed .env.example and no secrets in the repo; (d) structured JSON logging with a request-id correlation field on every log line; (e) /healthz liveness and /readyz readiness endpoints, where readiness reflects the actual state of dependencies rather than returning 200 unconditionally; (f) pytest configured with at least three passing tests and a coverage report; (g) GitHub Actions CI running lint (ruff), format check (black), type check (mypy),

	and tests on every push. The service should start, respond to /healthz, and pass CI before any feature code is written. • Perception service design doc (W01_Service_Design_Doc.docx): Write the design specification for the perception service. Cover: (a) the end-to-end flow from a frame leaving the robot to a detection reaching a consumer, drawn as a component diagram; (b) the component boundaries — what is the API layer's responsibility versus the worker's versus the store's; (c) the failure modes at each boundary and the intended behaviour for each (model not loaded, worker queue full, store unavailable, malformed payload, oversized payload); (d) the initial endpoint inventory for /v1/; (e) non-functional targets — latency budget, throughput target, maximum payload size. Apply the full-stack experience from the Stock Research Website: the design should be specific enough that the frontend or robot client could be built against it before the backend exists.
Deliverables	• W01_Backend_Platform_Brief.docx — 5-platform backend analysis; per-platform data profile, latency requirement, retention and privacy constraints, client shape, primary engineering challenge; three architecture decisions recorded in ADR format with justification. • W01_Service_Design_Doc.docx — component diagram, boundary responsibilities, failure-mode table, /v1/ endpoint inventory, non-functional targets. • W01_Service_Skeleton (GitHub repo) — FastAPI app running, /healthz and /readyz implemented, structured logging with request ids, pinned dependencies, .env.example, pytest suite green, GitHub Actions CI green. • weekly/Wk-01-Log.md — 300–500 words: which architecture decision was hardest, what the alternatives were, and what specifically decided it.
Tools	Python 3.11; FastAPI; Pydantic v2 + pydantic-settings; uvicorn; pytest + pytest-cov; ruff, black, mypy; GitHub + GitHub Actions; Mermaid or draw.io (component diagram); InGen public website for platform information; Word.
Self-Check	• Does the platform brief produce a different backend specification for each InGen platform — not one generic 'REST API' answer restated five times? • Is each architecture decision justified against a specific number from the data profile — expected payload size, frame rate, or latency budget — rather than a general preference? • Does /readyz actually check its dependencies, and does CI fail if a test fails? Verify by pushing a deliberately broken commit on a branch.

Week 2 · Sep 21–26, 2026 (Mon–Sat) · API Contract, Data Models & Frame Ingestion Path	
Platform Anchor	Aido Rover (outdoor patrol) · Phase A core API surface
Objective	Design and implement the versioned /v1/ API contract and the Pydantic data models. Build the frame ingestion path end to end with validation, size limits, and error handling — returning a stubbed detection response so that a client can integrate before the real model lands in Week 3.
Tasks	• Versioned API implementation: Implement the /v1/ endpoint set from the Week-1 inventory. Minimum surface: POST /v1/frames (accept a frame with robot_id, timestamp, and image payload; return a frame_id and either an inline result or a job reference, per the Week-1 execution-model decision); GET /v1/frames/{frame_id} (retrieve frame metadata and detection result or processing status); GET /v1/robots/{robot_id}/state (current encoded state — stubbed this week, real in Week 4); GET /v1/detections (list endpoint with filter parameters declared but not yet backed by the store). Every endpoint has explicit request and response Pydantic models, an explicit status code set, and an OpenAPI description that a client engineer could build against without asking a question. The auto-generated OpenAPI document at /docs is a deliverable, not a side effect. • Core data models and schema decisions: Define the core Pydantic v2 models: Frame (id, robot_id, platform, captured_at, received_at, width, height, format, size_bytes, checksum), Detection (id,

	frame_id, class_label, confidence, bounding box as normalised x/y/w/h with the coordinate convention stated explicitly, tracker_id nullable), and ErrorResponse (code, message, request_id, details). Decisions to make and document: normalised versus pixel coordinates (and why), timestamp representation (UTC ISO-8601 with timezone required — reject naive timestamps), confidence range and rounding, and how an unknown or low-confidence class is represented. From the Clash Royale state-encoding work: the schema decisions made here determine what a downstream decision policy can and cannot reason about, so they are worth an hour of thought each. • Ingestion hardening: validation, limits, and errors: Harden the ingestion path before it carries real load. Required: (a) a maximum request body size enforced at the framework level with a clear 413 response, not an unhandled crash; (b) content-type and image-decode validation returning 422 with a specific message — a corrupt JPEG must not reach the model; (c) a per-robot rate limit with a documented limit and a 429 response including a Retry-After header; (d) idempotency — a repeated submission of the same frame (matched on robot_id plus checksum plus captured_at) returns the original frame_id rather than creating a duplicate; (e) a consistent error envelope across every failure path, carrying the request id so a log line can be found from a client-side error. The Stock Research Website experience of integrating a third-party API is the relevant lens inverted: design the errors you would want to receive if you were the client. • Contract test suite: Write tests covering the contract, not just the happy path: a valid frame submission, an oversized payload, a corrupt image, a missing required field, a naive timestamp, a duplicate submission, and a rate-limit trip. Use FastAPI's TestClient with pytest fixtures. Target 80% line coverage on the API layer, and state the coverage number rather than the intention.
Deliverables	• W02_API_v1 (repo branch merged to main) — /v1/ endpoints implemented with request/response models, status codes, and OpenAPI descriptions; ingestion hardening (size limit, decode validation, rate limit, idempotency, error envelope) in place. • W02_API_Contract.md — the API contract as a standalone document: every endpoint, every field, every error code, the coordinate and timestamp conventions, and a worked curl example per endpoint. • W02_Test_Report — pytest output and coverage report showing the contract test suite passing, with the coverage percentage stated. • weekly/Wk-02-Log.md — 300–500 words: the schema decision most likely to be regretted in Week 5, and what was done to reduce that risk.
Tools	FastAPI (routing, dependencies, exception handlers); Pydantic v2; Pillow or OpenCV (image decode validation); slowapi or a custom middleware (rate limiting); pytest + TestClient + pytest-cov; OpenAPI/Swagger UI; curl or HTTPie.
Self-Check	• Can a client engineer implement against W02_API_Contract.md alone, without reading the source? Test this by writing the curl examples first and confirming each one works verbatim. • Does every error path return the same envelope shape with a request id — including the framework-generated 422 validation errors? • Does submitting the same frame twice return the same frame_id rather than creating a second row?

PHASE B · Perception Worker & State Encoding · Weeks 3–4 · Sep 28 – Oct 10 · Mid-Point EOW4

Week 3 · Sep 28 – Oct 3, 2026 (Mon–Sat) · YOLO Detection Worker, Inference Interface & Latency Budget

Platform Anchor	Aido Rover (outdoor patrol) · Detection worker v0.1 — the real model behind the Week-2 contract
Objective	Replace the stubbed detection response with a real YOLO detection worker behind a stable, versioned inference

	interface. Establish the measured latency budget. This is the week Calvin's Clash Royale vision pipeline becomes a service component.
Tasks	• Inference interface and two implementations: Define a Detector interface (abstract base class or Protocol) with a narrow surface: <code>load()</code>, <code>warm_up()</code>, <code>predict(image) -> list[Detection]</code>, and <code>unload()</code>. Implement it twice: a YoloDetector using Ultralytics YOLO with pretrained COCO weights, and a StubDetector returning deterministic fixtures. The stub is not a throwaway — it is what allows the API tests to run in CI without downloading model weights, and it is what a client integrates against when the GPU box is unavailable. The interface is selected by configuration, not by an import. This is a direct generalisation of the Clash Royale detection stage: there, the detector was the pipeline; here, the detector is a swappable component behind a contract. • Detection worker: lifecycle, queue, and timeouts: Implement the worker per the Week-1 execution-model decision. Required behaviours regardless of model: (a) the model is loaded once at startup and warmed with a dummy inference — the first real request must not pay the warm-up cost; (b) <code>/readyz</code> returns not-ready until the model is loaded and warm; (c) a bounded work queue with an explicit maximum depth, returning 503 with Retry-After when full rather than accepting work it cannot do; (d) a per-request inference timeout with a documented value, after which the request fails cleanly and the worker is checked for health; (e) graceful shutdown that drains in-flight work before exiting. Document the chosen queue depth and timeout values and the reasoning for each. • Model output to Detection mapping: Implement the mapping from raw model output to the Week-2 Detection model: coordinate conversion into the documented normalised convention, confidence thresholding with a configurable threshold, non-maximum suppression settings stated explicitly, class-label mapping from the model's index space to InGen-relevant labels (person, vehicle, bicycle, animal, obstacle, unknown), and explicit handling of the empty-detection case. The class mapping is a design decision with consequences: state which COCO classes map to which InGen label, which are dropped, and why. From the Clash Royale work, the lesson to carry: the mapping layer is where most silent bugs live, so it gets its own unit tests with fixed fixture images and expected outputs. • Latency measurement and budget document: Measure, do not estimate. Build a small script that runs N frames through the service and reports p50, p95, and p99 end-to-end latency, plus the decomposition into decode time, inference time, and serialisation time. Run it on at least two image resolutions and two batch settings. Record the numbers in the latency budget document alongside the Week-1 target, and state plainly whether the target is met. This is the same discipline as backtesting the stock model against a baseline: the measurement comes before the optimisation, and an unfavourable result is reported rather than tuned away.
Deliverables	• W03_Detection_Worker (merged) — Detector interface, YoloDetector and StubDetector implementations, worker lifecycle with warm-up, bounded queue, timeouts, graceful shutdown; output mapping with unit tests on fixture images. • W03_Latency_Budget.docx — measured p50/p95/p99 with the decode/inference/serialise decomposition, at two resolutions; hardware described; comparison against the Week-1 target with an explicit met/not-met statement. • W03_Detection_Samples — 10+ annotated sample outputs (input image, JSON detection response, rendered overlay) covering easy, cluttered, low-light, and empty-scene cases. • weekly/Wk-03-Log.md — 300–500 words: what transferred directly from the Clash Royale YOLOv10 pipeline, and what turned out to be different once the detector had to live behind an API.
Tools	Ultralytics YOLO (pretrained weights); PyTorch; OpenCV or Pillow; FastAPI BackgroundTasks / asyncio queue / multiprocessing (per the Week-1 decision); pytest; a small custom benchmark script; <code>time.perf_counter</code> for instrumentation.
Self-Check	• Does the API test suite still pass in CI without downloading model weights — proving the

	StubDetector is genuinely wired through configuration? • Is the first request after startup no slower than the hundredth — verifying that warm-up actually happens at load time? • Are the latency numbers measured with a script that can be re-run in Week 7 to produce comparable figures — not hand-timed once?
--	---

Week 4 · Oct 5–10, 2026 (Mon–Sat) · MID-POINT (Sat Oct 10) · Robot State Encoder, Fari Generalisation & Mid-Point Review	
Platform Anchor	Aido Rover (state encoding primary) · Fari (indoor companion — contract generalisation test) · Mid-Point review Saturday October 10
Objective	Build the robot state encoding layer that turns a stream of per-frame detections into a queryable model of what the robot currently believes about its surroundings. Validate the API contract against Fari's different requirements. Mid-point review Saturday October 10.
Tasks	• RobotState encoder (the core of the week): This is the direct descendant of the Clash Royale state-encoding work and the technical centre of the programme. Define and implement a RobotState model: (a) entity list — each tracked entity with a stable entity_id, class label, last-seen timestamp, last-known normalised position, confidence, and observation count; (b) simple temporal association across frames — match a new detection to an existing entity by class plus spatial proximity within a configurable threshold, otherwise create a new entity; document the association rule and its known failure cases (occlusion, fast motion, two identical objects adjacent); (c) staleness and eviction — an entity not observed for a configurable window is marked stale, then evicted, so state does not grow without bound; (d) aggregate fields — total entity count by class, most recent frame id, state version number incremented on every update; (e) a serialised JSON form that is the documented contract for downstream consumers. State the explicit non-goal: this is not a Kalman filter or a full multi-object tracker, and the documentation says so. • State endpoints and concurrency handling: Add GET /v1/robots/{robot_id}/state returning the current encoded state, and GET /v1/robots/{robot_id}/state/history returning the last N state versions. Concurrency matters here: two frames from the same robot may be processed simultaneously, so state updates need a per-robot lock or a single-writer design. Document the concurrency approach and add a test that submits frames concurrently and asserts the state is not corrupted. • Fari generalisation test and gap analysis: Take the API contract and the state schema and evaluate them against Fari's requirements from the Week-1 brief: close-range recognition, event-driven rather than continuous operation, and stricter retention limits on interior imagery. Produce a written gap analysis: which fields are Aido-Rover-specific, what a Fari client would need that the current contract does not offer, and what minimal change would make the contract serve both. Then implement the smallest change that closes the most important gap — typically a platform discriminator field and a per-platform retention policy hook. This is the test of whether the Week-2 contract was designed or merely written. • Mid-point review preparation: Prepare a 5-minute live demo for the Saturday October 10 mid-point call. Screen share and walk through: (1) the OpenAPI docs page and one live curl round trip; (2) a real frame submitted and a real detection returned, with the rendered overlay; (3) the state endpoint after several frames, showing entities appearing, updating, and going stale; (4) the CI run and coverage number. Iqbal will score across 7 rubric dimensions and confirm Weeks 5–8 scope.
Deliverables	• W04_State_Encoder (merged) — RobotState model, temporal association, staleness and eviction, aggregate fields, state and history endpoints, per-robot concurrency handling with a concurrency test. • W04_State_Schema.docx — the RobotState contract: every field, the association rule, the eviction policy, documented failure cases, and the explicit non-goals.

	• W04_Fari_Gap_Analysis.docx — 1–2 pages: what generalises, what does not, the minimal change implemented, and what is deferred with a reason. • Mid-point evaluation rubric (signed) — 7-dimension scoring; Weeks 5–8 scope confirmed. • weekly/Wk-04-Log.md — 300–500 words.
Tools	Pydantic v2; Python asyncio locks or threading primitives; pytest with concurrency tests (pytest-asyncio); FastAPI; Mermaid (state lifecycle diagram).
Self-Check	• Does the state actually stop growing? Run the service for an extended session and confirm entity count stabilises rather than climbing indefinitely. • Does the state schema document name its failure cases explicitly — or does it only describe the cases where the association rule works? • Did the Fari gap analysis change anything in the code, or did it conclude that no change was needed? The second conclusion is acceptable only if it is argued, not assumed.

PHASE C · Persistence, Query API & Event Pipeline · Weeks 5–6 · Oct 12 – Oct 24

Week 5 · Oct 12–17, 2026 (Mon–Sat) · Telemetry Store, Query API & Retention

Platform Anchor	Aido Rover (telemetry volume primary) · Fari (retention and privacy constraints)
Objective	Give the service a memory. Implement the telemetry store per the Week-1 storage decision, with a migrated schema, an indexed query API, and an enforced retention policy. This is the week the data-pipeline discipline from the stock analysis project applies directly.
Tasks	• Schema, migrations, and indexing: Implement the persistence layer with SQLAlchemy 2.0 and Alembic migrations — schema changes are migrations from the first commit, never hand-edited tables. Tables: robots, frames, detections, state_snapshots, and events (populated in Week 6). Decisions to make and document: (a) whether raw image bytes are stored, stored on disk with a path reference, or discarded after inference — argue it from the Week-1 retention and privacy analysis, not from convenience; (b) the primary and foreign key strategy; (c) which columns are indexed and why — at minimum (robot_id, captured_at) and (class_label, captured_at) to serve the Week-5 query patterns; (d) timestamp storage in UTC with the convention enforced at the model layer. The Purdue data structures grounding is the relevant lens: an index is a data-structure choice with a stated read/write trade-off, and the reasoning belongs in the ADR. • Query API with filtering, sorting, and pagination: Extend GET /v1/detections into a real query endpoint: filter by robot_id, class_label, minimum confidence, and a captured_at time range; sort by captured_at ascending or descending; paginate with a documented strategy. Choose cursor-based pagination over offset-based and justify it — offset pagination on a continuously-growing telemetry table skips and repeats rows under concurrent writes. Add GET /v1/robots/{robot_id}/timeline returning a time-bucketed detection summary for an operator view. Every filter combination gets a test, and every query is checked with EXPLAIN to confirm it uses an index rather than scanning the table. • Retention and purge job: Implement retention as scheduled, tested code, not a documented intention: a background job that deletes frames, detections, and state snapshots past a configurable per-platform age, with a dry-run mode that reports what would be deleted, structured logging of every purge run, and a test that inserts aged rows and asserts they are removed. Fari's interior imagery gets a shorter default retention than Aido Rover's outdoor telemetry, per the Week-1 privacy analysis. • Data-quality checks: Apply the data-cleaning discipline from the yfinance pipeline. Implement checks on inbound and stored data: timestamps in the future or implausibly far in the past, bounding boxes outside the [0,1] range, confidence values outside the valid range, frames arriving

	out of order for a robot, and duplicate detections within a frame. Each check either rejects with a specific error or records a data-quality counter. Expose the counters at a <code>/v1/admin/stats</code> endpoint. • Seed data generator: Write a seeding script that generates a realistic multi-day, multi-robot telemetry dataset so the query endpoints, the Week-6 dashboard, and the Week-7 benchmarks all have meaningful data to work against. Document how to run it.
Deliverables	• W05_Telemetry_Store (merged) — SQLAlchemy models, Alembic migrations, indexes, query endpoints with filtering/sorting/cursor pagination, retention job with dry-run, data-quality checks, <code>/v1/admin/stats</code>. • W05_Data_Model.docx — entity-relationship diagram, per-table column reference, index rationale, retention policy per platform, the raw-image storage decision and its privacy justification. • W05_Query_Examples.md — 8+ worked queries with the curl call, the response shape, and the EXPLAIN output confirming index use. • <code>scripts/seed_telemetry.py</code> — multi-day, multi-robot seed generator with usage documentation. • <code>weekly/Wk-05-Log.md</code> — 300–500 words: the indexing or pagination decision that mattered most, and the query it was measured against.
Tools	SQLAlchemy 2.0; Alembic; SQLite or PostgreSQL (per Week-1 ADR); APScheduler or a cron-invoked management command; pytest with a test database fixture; EXPLAIN / EXPLAIN QUERY PLAN; Faker or a custom generator for seed data.
Self-Check	• Does every schema change since Week 4 exist as an Alembic migration that runs cleanly from an empty database? • Does the timeline query use an index — confirmed by EXPLAIN output pasted into W05_Query_Examples.md, not assumed? • Does the retention job have a test that proves aged rows are deleted and recent rows are not?

Week 6 · Oct 19–24, 2026 (Mon–Sat) · Event & Alert Rule Engine, Webhooks & Operator Dashboard	
Platform Anchor	Sentinel Prime AI (alert consumer) · Aido Rover (event source) · Fari (event-driven interaction model)
Objective	Turn detections into decisions. Build a configurable event and alert rule engine, a webhook delivery path with retries, and a minimal operator dashboard served by the API — the frontend skill from the Stock Research Website applied to a live operational view.
Tasks	• Event and alert rule engine: Implement a rule engine evaluating declarative rules against incoming detections and state updates. A rule is data, not code: name, enabled flag, platform or robot scope, condition (<code>class_label</code> in a set, minimum confidence, optional spatial zone as a normalised bounding region, optional time-of-day window, optional minimum consecutive-frame count to suppress single-frame noise), severity, and cooldown period. Rules are stored in the database and manageable through <code>/v1/rules</code> (list, create, update, disable) — an operator must be able to add a rule without a deployment. Evaluation produces an Event row with the rule id, triggering detection ids, severity, and timestamp. The consecutive-frame and cooldown requirements exist because a naive rule engine that fires on every frame produces an unusable alert stream; state that reasoning in the documentation. • Webhook delivery with signing and retries: Implement outbound webhook delivery for events: a registered endpoint per subscriber, a signed payload (HMAC-SHA256 over the body with a shared secret, signature in a header) so the receiver can verify authenticity, exponential backoff with jitter on failure, a maximum retry count, and a dead-letter table for permanently failed deliveries with an admin endpoint to inspect it. Delivery is asynchronous and must never block the ingestion path. This is the Stock Research Website's third-party integration experience inverted: this time

	Calvin is the upstream, and the contract he offers is the one someone else has to integrate against. • Operator dashboard (GET /dashboard): Build a single-page operator view served by the service: a live list of recent detections with class and confidence, per-robot current state with entity counts, an active-events panel with severity colouring, and a simple time-range filter hitting the Week-5 query API. Plain HTML, CSS, and vanilla JavaScript with polling or server-sent events — no build step, no framework, because a dashboard that requires a Node toolchain to run is a dashboard that will not be run. This is a deliberate reuse of the frontend stack Calvin already shipped with, applied to an operational rather than a consumer surface. • Event query and acknowledgement endpoints: Add GET /v1/events with the same filtering and pagination conventions established in Week 5, and POST /v1/events/{event_id}/acknowledge to record operator acknowledgement with an actor and a timestamp. Consistency with the Week-5 conventions is itself a deliverable: a client that learned the detections query should not have to learn a second pattern for events.
Deliverables	• W06_Event_Engine (merged) — rule model and CRUD endpoints, evaluation pipeline with consecutive-frame and cooldown suppression, event persistence, webhook delivery with HMAC signing, backoff, retries, and dead-letter handling. • W06_Dashboard — single-page operator view served at /dashboard: live detections, per-robot state, active events, time-range filter; no build step required. • W06_Event_Documentation.docx — rule specification and field reference, three worked example rules (patrol intrusion, unattended-vehicle, Fari interaction event), webhook payload schema, signature verification instructions with a code sample a subscriber can copy. • W06_Demo_Recording.mp4 — 60–90 seconds: frames streaming in, detections appearing on the dashboard, a rule firing, an event acknowledged, and the webhook received by a test endpoint. • weekly/Wk-06-Log.md — 300–500 words: what made the alert stream usable rather than noisy, and the specific suppression rule that did it.
Tools	FastAPI; SQLAlchemy; httpx (async webhook delivery); hmac/hashlib (payload signing); tenacity or a hand-rolled backoff; vanilla HTML/CSS/JavaScript (dashboard); server-sent events or polling; webhook.site or a local receiver for delivery testing.
Self-Check	• Can a new rule be added through the API and take effect without restarting the service — verified live, not assumed? • Does a webhook receiver reject a payload whose signature has been tampered with, using only the documented verification instructions? • Does the dashboard open and function from a clean checkout with no npm install and no build step?

PHASE D · Reliability, Performance & Capstone Release · Weeks 7–8 · Oct 26 – Nov 7 · Final Readout Mon Nov 9

Week 7 · Oct 26–31, 2026 (Mon–Sat) · Load Testing, Profiling, Observability & Containerisation	
Platform Anchor	Full service under simulated multi-robot load · Aido Rover fleet scenario
Objective	Establish what the service can actually take, fix what the measurements expose, and make it deployable and observable. Measure first, optimise second, report honestly — the same order used to backtest the stock model.
Tasks	• Load test harness and baseline runs: Build a reproducible load harness with Locust or k6 modelling a realistic fleet: N simulated Aido Rovers each submitting frames at a fixed rate for a fixed duration, with a ramp phase and a steady-state phase. Scenarios to run: 1 robot (baseline), 5 robots, 10 robots, and a burst scenario where a robot reconnects and submits a backlog. Record

	for each: throughput in frames per second, p50/p95/p99 end-to-end latency, error rate by status code, queue depth over time, CPU and memory. Run every scenario before optimisation and record it as the baseline — an optimisation without a baseline is an anecdote. • Profiling and targeted optimisation: Profile the paths the load test identifies as slow using cProfile and py-spy, and memory behaviour with tracemalloc or memray. Expect and specifically investigate: redundant image decode or re-encode on the frame path, unnecessary copies of image buffers (the Advanced Programming in C grounding is directly relevant — know how many times the payload is copied and why), N+1 query patterns in the detection and event list endpoints, synchronous work blocking the event loop, and per-request model or database connection setup that should be pooled. Fix what the profile shows, not what intuition suggests, and record the measured delta for each change individually. • Observability: metrics, tracing context, and runbook: Instrument the service so an on-call engineer can diagnose it without attaching a debugger: (a) Prometheus metrics at /metrics — request count and duration histograms by endpoint and status, inference duration histogram, queue depth gauge, webhook delivery success and failure counters, data-quality counters; (b) request-id propagation from ingress through the worker into every log line and into the error response; (c) log levels used correctly, with the noisy per-frame logging at DEBUG and the operationally meaningful events at INFO or WARNING; (d) a documented runbook section listing the three most likely production failures — model fails to load, queue saturation, database unavailable — with the symptom, the metric or log line that identifies it, and the first action. • Containerisation and compose stack: Produce a multi-stage Dockerfile (build stage with toolchain, slim runtime stage, non-root user, no secrets baked into layers) and a docker-compose.yml bringing up the API, the database, and the worker together, plus optional Prometheus. Verify the whole stack from a clean clone on a machine that has never run the project: docker compose up, then a successful frame submission, in under five commands. Document those commands exactly. • Performance report: Re-run every load scenario post-optimisation and write the report: baseline versus optimised for each metric and scenario, the individual measured contribution of each optimisation, the hardware and methodology so the numbers are reproducible, and — required, not optional — the bottlenecks that remain unresolved, with a technical explanation of what would be needed to address them.
Deliverables	• W07_Load_Harness — Locust or k6 scenarios (1, 5, 10 robots, burst reconnect) as committed, re-runnable code with usage documentation. • W07_Performance_Report.docx — baseline versus optimised tables for throughput, p50/p95/p99 latency, error rate, and resource use across all scenarios; per-optimisation measured deltas; methodology and hardware; remaining bottlenecks named with technical explanation. • W07_Observability (merged) — /metrics endpoint, request-id propagation end to end, corrected log levels, runbook section with three failure modes. • Dockerfile + docker-compose.yml — multi-stage build, non-root runtime, full stack up from a clean clone in five documented commands. • weekly/Wk-07-Log.md — 300–500 words: the optimisation that produced the largest measured gain, and the technical reason it worked.
Tools	Locust or k6; cProfile, py-spy, snakeviz; tracemalloc or memray; prometheus-client; Docker + Docker Compose; Prometheus (optional local); pytest for regression coverage after each optimisation.
Self-Check	• Is every performance claim backed by a number from a scenario that can be re-run by someone else on their own hardware? • Does the full stack come up from a clean clone with no manual steps beyond the documented commands — tested on a fresh directory? • Does the report name at least one thing that did not get fixed, and explain why?

Week 8 · Nov 2–7, 2026 (Mon–Sat) + Final Readout Mon Nov 9 · Capstone Repository, Technical Deck & Final Readout

Platform Anchor	Full ingen_perception_service_v1.0 package — API + detection worker + state encoder + telemetry store + event engine + dashboard + containerised deployment
Objective	Finalise and package the complete ingen_perception_service_v1.0 repository. Write the comprehensive README. Build the 10-slide capstone deck. Deliver the final readout on Monday November 9. Note: US daylight saving ends Sunday November 1, so this week's syncs run at 09:00 EST / 19:30 IST.
Tasks	• ingen_perception_service_v1.0 repository: Organise the repository as a clean, public-ready deliverable. Requirements: (a) no secrets in the code or in the git history — verify with a secret scanner such as gitleaks and state that the scan was run and passed; (b) a root README.md covering project overview, architecture diagram, system requirements, quickstart (numbered steps, each with an expected outcome), the full API reference or a link to the served OpenAPI docs, the state schema, the rule specification, configuration reference with every environment variable and its default, performance benchmarks, and known limitations; (c) docs/adr/ containing the architecture decision records from Weeks 1, 3, and 5 in their original form — including any decision later revised, with the revision noted rather than the record rewritten; (d) CONTRIBUTING.md with the development setup, test command, and lint command; (e) LICENSE (MIT) and a NOTICE file attributing the YOLO weights and any third-party assets with their licences; (f) a GitHub Release tagged v1.0.0 with generated release notes; (g) CI green on main, with the badge in the README. Apply the operational discipline from 16 Handles: the repository should be runnable by someone on a different shift with no handover conversation. • Capstone deck (W08_Capstone_Deck.pdf, 10 slides): A 10-slide technical presentation for a mixed audience of InGen leadership and engineering. Slide structure: (1) Title and programme overview. (2) The problem: what a perception backend must do for an InGen robot, and why latency, contract stability, and queryability are the three axes that matter. (3) Architecture: the component diagram, with the three Week-1 decisions and what each one bought. (4) The API contract: one endpoint shown end to end, and what makes it stable enough for a client team to build against. (5) Detection worker: the interface, the swappable stub, and the measured latency budget. (6) State encoding: from a stream of detections to a queryable robot state — the technical centre of the work, with the association rule and its honest limitations. (7) Telemetry and query: the data model, the index decisions, and a live query result. (8) Events and alerts: the rule engine, the suppression logic that made the alert stream usable, and the dashboard. (9) Performance: baseline versus optimised table, the largest single win, and the remaining bottleneck. (10) Next steps: three specific technical developments — for example gRPC or WebSocket transport for the humanoid control loop, multi-object tracking to replace the proximity association rule, and horizontal scaling of the worker tier behind a shared queue. Every slide title states a finding or outcome, not a topic. • Retrospective (W08_Retrospective.docx): One page covering: (a) the engineering decision in this programme that Calvin would make differently with hindsight, and what specifically he learned that changed his view; (b) the connection between the Clash Royale AI Agent pipeline and this service — what transferred directly, and what turned out to be genuinely different once the perception loop had to survive concurrent clients, failures, and a stable public contract; (c) career intent: what this programme reveals about where he wants to take the intersection of ML systems and backend engineering, given the Purdue AI/ML concentration and a 2028 graduation date. • Final readout: Monday November 9, 09:00 EST / 19:30 IST. A 30–40 minute presentation with a live screen-shared demo of the running stack, followed by Q&A with Iqbal. Prepare to defend: the three Week-1 architecture decisions and whether the measurements later vindicated them; the state association rule and its failure cases; the pagination and indexing choices in Week 5; and the specific numbers in the performance report, including the methodology behind them.
Deliverables	• ingen_perception_service_v1.0 (GitHub repository) — clean history, secret scan passed, comprehensive README, ADRs preserved, CONTRIBUTING, LICENSE and NOTICE, GitHub Release tagged v1.0.0, CI green with badge.

	• W08_Capstone_Deck.pdf — 10 slides; finding-statement title on every slide; screen-share ready with a shareable repository link. • W08_Retrospective.docx — 1 page; hindsight decision; Clash Royale to production-service skill transfer; career intent. • Final evaluation rubric (signed) — 7-dimension scoring; career note from Iqbal.
Tools	Git / GitHub (Releases, Actions); gitleaks or trufflehog (secret scan); Markdown; Mermaid (architecture diagram); PowerPoint, Google Slides, or Figma; PDF export.
Self-Check	• Can a new engineer go from clone to a successful frame submission using only the README quickstart — tested on a clean machine or a fresh container? • Is the v1.0.0 release tagged, with release notes, and does the linked artifact match the tagged commit? • Is the 'next steps' slide three named technical developments each with a stated rationale — not generic 'improve performance and add features'?

4. Deliverables Index

All deliverables committed to the ingen_perception_service_v1.0 GitHub repository or shared as organised documents. Primary output: the ingen_perception_service_v1.0 GitHub Release. Portfolio artifact: the repository link plus the capstone deck.

Wk	Artifact	Description	Due
1	W01_Backend_Platform_Brief.docx	Five-platform backend analysis: data profile, latency requirement, retention and privacy constraints, client shape, primary engineering challenge; three architecture decisions in ADR format	Sat Sep 19
1	W01_Service_Design_Doc.docx	Component diagram, boundary responsibilities, failure-mode table, /v1/ endpoint inventory, non-functional targets	Sat Sep 19
1	W01_Service_Skeleton (repo)	FastAPI app, /healthz and /readyz, structured logging with request ids, pinned dependencies, .env.example, pytest green, CI green	Sat Sep 19
1	weekly/Wk-01-Log.md	300–500 word log: hardest architecture decision and what decided it	Sat Sep 19
2	W02_API_v1 (merged)	/v1/ endpoints with request/response models; size limits, decode validation, rate limiting, idempotency, consistent error envelope	Sat Sep 26
2	W02_API_Contract.md	Standalone contract: endpoints, fields, error codes, coordinate and timestamp conventions, worked curl example per endpoint	Sat Sep 26
2	W02_Test_Report	Contract test suite output and coverage report with the percentage stated	Sat Sep 26
2	weekly/Wk-02-Log.md	300–500 word log	Sat Sep 26
3	W03_Detection_Worker (merged)	Detector interface, YoloDetector and StubDetector, warm-up, bounded queue, timeouts, graceful shutdown, output mapping with unit tests	Sat Oct 3
3	W03_Latency_Budget.docx	Measured p50/p95/p99 with decode/inference/serialise decomposition at two resolutions; met/not-met against the Week-1 target	Sat Oct 3
3	W03_Detection_Samples	10+ annotated samples: input, JSON response, rendered overlay; easy, cluttered, low-light, empty-scene cases	Sat Oct 3
3	weekly/Wk-03-Log.md	300–500 word log	Sat Oct 3
4	W04_State_Encoder (merged)	RobotState model, temporal association, staleness and eviction, state and history endpoints, per-robot concurrency handling	Sat Oct 10
4	W04_State_Schema.docx	Field reference, association rule, eviction policy, documented failure cases, explicit non-goals	Sat Oct 10

Wk	Artifact	Description	Due
4	W04_Fari_Gap_Analysis.docx	1–2 pages: what generalises, what does not, minimal change implemented, deferred items with reasons	Sat Oct 10
4	Mid-point evaluation rubric (signed)	7-dimension scoring; written feedback from Iqbal; Weeks 5–8 scope confirmed	Sat Oct 10
4	weekly/Wk-04-Log.md	300–500 word log	Sat Oct 10
5	W05_Telemetry_Store (merged)	SQLAlchemy models, Alembic migrations, indexes, query endpoints with cursor pagination, retention job, data-quality checks, admin stats	Sat Oct 17
5	W05_Data_Model.docx	ER diagram, column reference, index rationale, per-platform retention policy, raw-image storage decision with privacy justification	Sat Oct 17
5	W05_Query_Examples.md	8+ worked queries with curl call, response shape, and EXPLAIN output confirming index use	Sat Oct 17
5	scripts/seed_telemetry.py	Multi-day, multi-robot telemetry seed generator with usage documentation	Sat Oct 17
5	weekly/Wk-05-Log.md	300–500 word log	Sat Oct 17
6	W06_Event_Engine (merged)	Rule model and CRUD, evaluation with consecutive-frame and cooldown suppression, event persistence, signed webhooks with backoff and dead-letter	Sat Oct 24
6	W06_Dashboard	Operator view at /dashboard: live detections, per-robot state, active events, time-range filter; no build step	Sat Oct 24
6	W06_Event_Documentation.docx	Rule specification, three worked example rules, webhook payload schema, signature verification with a copyable code sample	Sat Oct 24
6	W06_Demo_Recording.mp4	60–90 seconds: ingestion, detection, rule firing, acknowledgement, webhook received	Sat Oct 24
6	weekly/Wk-06-Log.md	300–500 word log	Sat Oct 24
7	W07_Load_Harness	Locust or k6 scenarios (1, 5, 10 robots, burst reconnect) as re-runnable committed code with usage docs	Sat Oct 31
7	W07_Performance_Report.docx	Baseline versus optimised across all scenarios; per-optimisation measured deltas; methodology and hardware; remaining bottlenecks named	Sat Oct 31
7	W07_Observability (merged)	/metrics endpoint, request-id propagation, corrected log levels, runbook with three failure modes	Sat Oct 31
7	Dockerfile + docker-compose.yml	Multi-stage build, non-root runtime; full stack up from a clean clone in five documented commands	Sat Oct 31
7	weekly/Wk-07-Log.md	300–500 word log	Sat Oct 31
8	ingen_perception_service_v1.0/ (GitHub repo)	Complete service: API, worker, state encoder, telemetry store, event engine, dashboard, deployment; README, ADRs, LICENSE and NOTICE, Release v1.0.0	Sat Nov 7
8	W08_Capstone_Deck.pdf	10-slide technical capstone: problem, architecture, contract, worker, state encoding, telemetry, events, performance, next steps	Sat Nov 7
8	W08_Retrospective.docx	1 page: hindsight decision; Clash Royale to production-service transfer; career intent	Sat Nov 7
8+	Final readout (Mon Nov 9)	30–40 min deck presentation and live demo + Q&A with Iqbal · 09:00 EST / 19:30 IST	Mon Nov 9
8+	Final evaluation rubric (signed)	7-dimension final scoring; career note from Iqbal	Mon Nov 9

5. Working Norms — Remote, ET / IST

Calvin is on US Eastern Time (EDT, UTC-4 through October 31; EST, UTC-5 from November 1). InGen supervisor Iqbal Patel is on IST (UTC+5:30). The offset is 9.5 hours, becoming 10.5 hours for the final two weeks after US daylight saving ends on Sunday November 1. That offset is workable but leaves only a narrow natural overlap, so the programme is designed for async-first operation. Saturday syncs at 09:00 ET, which is 18:30 IST before November 1 and 19:30 IST after. Calvin is available for interviews and calls at any reasonable hours Eastern Time, which gives flexibility to move a sync earlier in his morning if a later IST slot is preferable.

5.1 Checkpoint Cadence

Checkpoint	When	Format	Scope
Day 1	Mon Sep 14	Async note (09:00 ET)	Calvin reads the plan in full and sends a Day-1 async note: plan understood, initial view on the three Week-1 architecture decisions, environment set up, any blockers identified. Programme begins in full.
Daily Async	Mon–Sat	Daily note (09:00 ET)	Yesterday / Today / Blockers. Technical questions are specific: 'I cannot confirm the expected frame rate for Aido Rover from public information — I am proceeding with 5 fps at 1280×720 as the design assumption, which sets the ingestion target at 25 frames/sec across a 5-robot fleet; please confirm or correct.' Not: 'How should I design the ingestion?'
Wk-2 Sync	Sat Sep 26	30 min live	API contract walkthrough; OpenAPI docs reviewed live; Weeks 3–4 scope confirmed. 09:00 EDT / 18:30 IST.
Wk-3 Sync	Sat Oct 3	20–30 min	Detection worker demo; latency budget findings reviewed; any scope adjustment before mid-point.
Mid-Point Review	Sat Oct 10	45 min (formal)	Full rubric scored across 7 dimensions. Live demo: OpenAPI docs, real frame in and detection out, state endpoint over successive frames, CI and coverage. Weeks 5–8 scope confirmed. 09:00 EDT / 18:30 IST.
Wk-5 Sync	Sat Oct 17	20–30 min	Data model and query API review; index rationale and retention policy checked.
Wk-6 Sync	Sat Oct 24	20–30 min	Event engine and dashboard demo; webhook contract review; documentation completeness check.
Wk-7 Sync	Sat Oct 31	30 min	Performance report findings; container stack verified live from a clean clone; capstone deck outline confirmed.
Final Readout	Mon Nov 9	30–40 min	Capstone deck presentation and live demo + Q&A. Final rubric signed. Career note. 09:00 EST / 19:30 IST.

5.2 Day-to-Day Rhythm

- Daily async note (09:00 ET each working day): Yesterday / Today / Blockers. Every technical blocker includes a stated default assumption so that work continues through the 9.5-hour response gap: 'I cannot determine InGen's required retention period for Fari interior imagery. I am proceeding with a 7-day default and a per-platform configuration key so the value can be changed without a code change. Please confirm or correct.' Work does not stop for a blocker; the assumption is documented and made easy to reverse.
- Weekly log (Saturday, 300–500 words): one specific technical decision — the alternatives considered and the reasoning for the choice. 'I chose cursor pagination over offset pagination for /v1/detections because the table receives continuous inserts, and offset pagination would cause an operator paging backwards through a live feed to see repeated and skipped rows' is a decision log. 'I worked on the query API this week and it is going well' is a status update.
- Every merged pull request is small enough to review in one sitting, has a description stating what changed and why, and leaves CI green. Even working solo, the PR discipline is what makes the repository legible later — and it is what the mid-point and final rubric read.
- Demo recordings from Week 3 onward: each weekly deliverable includes a 30–60 second screen recording of the new capability actually running. For a backend service, a short recording of a live request round trip communicates working software far more convincingly than a passing test count.

- 9.5-hour offset management: a note sent at 09:00 ET arrives at 18:30 IST, near the end of Iqbal's working day, so a reply typically lands overnight and is waiting when Calvin starts the next morning. Plan for a 24-hour cycle on technical questions and never hold work for more than one cycle. From November 1 the offset becomes 10.5 hours; the Saturday sync moves to 19:30 IST and the daily rhythm is otherwise unchanged.
- Purdue coursework runs concurrently with this internship. The 6-day structure assumes a sustainable weekday load with Saturday reserved for the weekly deliverable, log, and sync. If an exam week compresses availability, that is flagged in the daily note at least a week ahead and scope is adjusted in the sync — not discovered at the deliverable deadline.

5.3 Quality Standards

- Repository hygiene: CI green on main at all times; lint, format, and type checks enforced rather than advisory; no secrets in code or history; every schema change is a migration; dependencies pinned. The repository should be cloneable and runnable by an engineer who has never seen it.
- Contract stability: once an endpoint is published in a weekly deliverable, breaking changes require a documented migration note in the weekly log. The API contract is a promise to a client team, not a draft.
- Measured, not estimated: every performance claim in this programme comes from a re-runnable script whose methodology and hardware are documented. Latency targets, throughput figures, and optimisation gains are numbers with a provenance.
- Documented failure behaviour: every component states what it does when its dependency is unavailable — model not loaded, queue full, database down, webhook receiver unreachable. A service whose failure behaviour is undocumented is a service that has not been designed.
- Privacy and licensing: no proprietary InGen data, no unlicensed model weights, no images of identifiable people without consent. Retention limits on interior imagery are enforced in code, not stated in a document.

6. Evaluation Rubric — Mid-Point & Final

The same 7-dimension rubric is applied at the end of Week 4 (mid-point, Saturday October 10) and at the Week 8 final readout (Monday November 9). Total: 35 points.

6.1 Scoring Scale

Score	Label	Definition
5	Junior SDE Grade — Production-Ready Service for a Robot Fleet	ingen_perception_service_v1.0 is a deployable backend: it starts from a clean clone with the documented commands, meets its stated latency budget under multi-robot load, exposes a stable versioned contract a client team could build against, persists and serves queryable telemetry with enforced retention, and ships with a README an engineer can follow without assistance. The capstone deck communicates the engineering work clearly to a non-engineer audience. Could be submitted as a portfolio project for a backend or ML-systems engineering role at a robotics or infrastructure company.
4	Strong	All core deliverables functional. API contract stable and documented. Detection worker and state encoder correct with measured latency. Telemetry and event pipeline working with tests. Performance report evidence-based. Minor gaps in documentation depth or in the secondary (Fari) generalisation.
3	Solid	Core service functional end to end. Some reliability or performance work outstanding. Event pipeline or dashboard partially complete. Test coverage present but uneven. Documentation present but needs expansion. No major rework required.
2	Developing	Below standard on one or more rubric dimensions. Corrective feedback documented.
1	Needs Improvement	Service does not run from a clean clone; contract undocumented or unstable; no measured performance data; tests absent or failing; deliverables incomplete. Significant rework required.

6.2 Rubric Dimensions (7 dimensions, 35 points)

Dimension	What Is Being Evaluated	Mid /5	Final /5
API & Contract Design	The /v1/ surface is coherent, versioned, and consistent: the same conventions for pagination, filtering, errors, and timestamps across every endpoint. Request and response models are explicit. The OpenAPI document and W02_API_Contract.md are sufficient for a client engineer to integrate without asking a question. Breaking changes after publication are documented, not silent. Error responses are specific and carry a request id.	__ / 5	__ / 5
Code Quality & Testing	Clear module boundaries between API, service, worker, and persistence layers — no business logic in route handlers, no database access from the worker. Type hints throughout with mypy passing. Tests cover failure paths as well as happy paths, including oversized payloads, corrupt input, concurrent state updates, and retention deletion. CI is green and enforces lint, format, type check, and tests. Coverage is measured and stated.	__ / 5	__ / 5
ML Systems Integration	The detection worker treats the model as a swappable component behind a stable interface, not as an inlined dependency: warm-up at startup, bounded queue with explicit backpressure, per-request timeout, and a stub implementation that lets CI run without weights. The model output mapping — coordinates, thresholds, class mapping — is documented and unit-tested. Latency is measured and decomposed, not estimated. The Clash Royale perception pipeline is visibly evolved into a service component, not merely repeated.	__ / 5	__ / 5
Data Modelling & State Encoding	The RobotState encoder produces a representation a downstream decision policy could actually act on: stable entity identity across frames, honest confidence handling, bounded growth through staleness and eviction, and documented failure cases for the association rule. The telemetry schema is migration-managed, correctly indexed for its real query patterns, and enforces its retention policy in code. Index and pagination choices are justified against a measured query, not asserted.	__ / 5	__ / 5
Reliability & Operability	Failure behaviour is designed and documented at every boundary: model unavailable, queue saturated, database down, webhook receiver unreachable. Health and readiness endpoints reflect real dependency state. Metrics, structured logs, and request-id propagation let an engineer diagnose a problem without a debugger. The service comes up from a clean clone via the documented commands. The runbook names the three most likely failures with their symptom and first action.	__ / 5	__ / 5
Technical Documentation & Communication	Every deliverable has its corresponding document: the API contract, the state schema, the data model, the event and webhook specification, the performance report, and the ADRs preserved in original form with revisions noted. The README takes a new engineer from clone to a working request. The capstone deck translates backend engineering work into implications for InGen's robot platforms for a mixed technical and non-technical audience.	__ / 5	__ / 5
Remote Ownership & Async Professionalism	ET/IST is a 9.5-hour offset, widening to 10.5 hours after November 1. Daily async notes land by 09:00 ET each working day. Blockers are flagged within 24 hours and always include a stated default assumption so work does not stop. Weekly logs document a specific technical decision with alternatives and reasoning — not a task list. Saturday syncs are prepared in advance: Calvin arrives with a status summary, a working demo, and a specific agenda item. Concurrent Purdue coursework pressure is flagged ahead of time, not after a missed deliverable.	__ / 5	__ / 5
TOTAL (out of 35)		__ / 35	__ / 35

7. Repository Structure — ingen_perception_service_v1.0

The ingen_perception_service_v1.0 repository is the primary portfolio deliverable. It should be cloneable and runnable by any Python engineer with Docker installed, using only the README quickstart. Model weights are downloaded at build or first run rather than committed; large fixture media use Git LFS.

Path	Contents
README.md	Project overview, architecture diagram, requirements, numbered quickstart with expected outcomes, API reference, state schema, rule specification, configuration reference, benchmarks, known limitations, next steps
app/api/v1/	Route modules: frames.py · detections.py · robots.py · events.py · rules.py · admin.py — routing and validation only
app/models/	Pydantic request/response schemas: frame.py · detection.py · robot_state.py · event.py · errors.py
app/services/	Business logic: state_encoder.py · rule_engine.py · retention.py · webhook_dispatcher.py
app/workers/	detector.py (Detector interface) · yolo_detector.py · stub_detector.py · queue.py (bounded work queue, lifecycle, warm-up)
app/db/	SQLAlchemy models, session management, repository classes; alembic/ migration history
app/core/	config.py (pydantic-settings) · logging.py (structured JSON, request-id) · metrics.py (Prometheus) · security.py (HMAC webhook signing)
static/dashboard/	Operator dashboard: index.html · dashboard.js · dashboard.css — no build step required
tests/	unit/ · integration/ · contract/ · fixtures/ (sample frames and expected detection outputs)
load/	Locust or k6 scenarios: baseline_1_robot · fleet_5 · fleet_10 · burst_reconnect, with usage documentation
scripts/	seed_telemetry.py · benchmark_latency.py · run_migrations.sh
docs/adr/	0001-storage-engine.md · 0002-inference-execution-model.md · 0003-transport.md · 0004-pagination-strategy.md — original decisions preserved, revisions noted
docs/	W01_Backend_Platform_Brief.pdf · W02_API_Contract.md · W04_State_Schema.pdf · W05_Data_Mode l.pdf · W06_Event_Documentation.pdf · W07_Performance_Report.pdf · W08_Capstone_Deck.pdf
weekly/	Wk-01-Log.md through Wk-08-Log.md
Dockerfile · docker-compose.yml	Multi-stage build with non-root runtime; compose stack bringing up API, database, worker, and optional Prometheus
.github/workflows/ci.yml	Lint (ruff) · format check (black) · type check (mypy) · pytest with coverage, on every push and pull request
.env.example · LICENSE · NOTICE · CO NTRIBUTING.md · .gitignore	Configuration template with no real values; MIT licence; third-party attributions including YOLO weights; development setup and commands; Python .gitignore

8. Toolchain

Category	Tools / Resources
Language & Runtime	Python 3.11; type hints throughout; uv or Poetry for dependency and lockfile management
Web Framework	FastAPI; uvicorn; Pydantic v2 and pydantic-settings; auto-generated OpenAPI / Swagger UI
ML Serving	Ultralytics YOLO with pretrained weights; PyTorch; OpenCV or Pillow for decode and preprocessing
Persistence	SQLAlchemy 2.0; Alembic migrations; SQLite or PostgreSQL per the Week-1 ADR
Async & Messaging	asyncio; bounded queues; httpx for outbound webhooks; tenacity or hand-rolled exponential backoff with jitter
Testing	pytest; pytest-asyncio; pytest-cov; FastAPI TestClient; fixture images with expected detection outputs
Quality Gates	ruff (lint); black (format); mypy (type check); GitHub Actions CI; gitleaks or trufflehog (secret scan)
Load & Profiling	Locust or k6; cProfile; py-spy; snakeviz; tracemalloc or memray
Observability	prometheus-client; structured JSON logging with request-id correlation; optional local Prometheus via compose

Category	Tools / Resources
Frontend (dashboard)	Vanilla HTML, CSS, and JavaScript; server-sent events or polling; no build toolchain
Deployment	Docker multi-stage build; Docker Compose; non-root runtime user; environment-variable configuration
Version Control	Git + GitHub; small reviewable pull requests; GitHub Releases; Git LFS for large fixture media
Documentation	Markdown (README, ADRs, contract); Mermaid or draw.io (diagrams); Microsoft Word; PDF export
Capstone Deck	Microsoft PowerPoint, Google Slides, or Figma; exported to PDF

9. Closing Note from the Supervisor

Calvin arrives at InGen with an unusually complete match to what this programme asks for — not because his résumé lists a robotics internship, but because the project he cofounded is structurally the same system, at a smaller scale, with the production requirements removed. The Clash Royale AI Agent takes a live visual stream, runs a fine-tuned YOLOv10 detector over it, transforms those raw detections into a structured state encoding, and hands that state to a decision policy trained through self-play. That is the perception service in miniature. What this programme adds is everything that changes when the pipeline stops being a single process on one machine and becomes a service several clients depend on: a versioned contract, concurrency, backpressure, persistence, retention, failure behaviour, and measured latency under load.

The second-year timing is a deliberate feature of this plan, not a limitation to work around. Calvin is a rising sophomore in Computer Engineering with Data Structures and Advanced Programming in C behind him and a 3.94 GPA, which means the fundamentals are solid but the production-engineering vocabulary — architecture decision records, migrations, backpressure, cursor pagination, observability, runbooks — is largely new. The programme is therefore front-loaded with decisions rather than code: Week 1 is spent choosing a storage engine, an inference execution model, and a transport, and writing down why. That is the part of engineering that is hardest to learn from coursework and hardest to fake in a review.

The Stock Research Website is the reason Week 1 can start at a production standard rather than a tutorial standard. Calvin has already shipped a FastAPI backend with a browser frontend, integrated a third-party market API he did not control, and wired an LLM into a user-facing flow. He does not need to learn what a route handler is. He needs to learn why the route handler should contain no business logic, why the third-party call needs a timeout and a documented fallback, and why the error he returns matters as much as the success. Those are corrections to an existing skill, which is a much faster path than instruction from zero.

The stock analysis model contributes the habit this programme leans on hardest in Week 7. Calvin built a model, tuned it, and then backtested it against a random baseline on SPY data rather than reporting only the favourable slices. An intern who has already compared his own work against a baseline understands, without being told, why the load test runs before the optimisation and why the performance report must name the bottleneck that was not fixed. Most of the value in Week 7 comes from that ordering, and it is already a habit rather than a rule imposed on him.

The two years managing 16 Handles are not a software credential and are not treated as one, but they are the reason the async operating model in Section 5 is a reasonable expectation. The ET/IST offset is 9.5 hours, widening to 10.5 after November 1, which leaves a narrow natural overlap and makes the daily note the primary instrument of supervision. Someone who has run scheduling and inventory for a store has already learned to report status before being asked and to flag a shortage while there is still time to act on it. The requirement that every blocker carries a stated default assumption — so that work continues through a 24-hour response cycle — is asking for a discipline he has practised in a different setting.

The mid-point on Saturday October 10 is the real decision point. By then the API contract, the detection worker, and the state encoder must all exist and work together, because Weeks 5 through 8 build entirely on top of them. If the contract is unstable or the state encoder is not honest about its failure cases at mid-point, the correct response is to narrow the Weeks 5–8 scope rather than to accumulate features on an unsound foundation, and that adjustment is an explicit agenda item at the mid-point review.

The career note at the final review will address what `ingen_perception_service_v1.0` reveals about Calvin's trajectory. The AI/ML concentration at Purdue and a 2028 graduation date leave considerable room, and the interesting question is not whether he becomes an ML engineer or a backend engineer, but whether he continues to work at the seam between them

— model serving, inference infrastructure, robotics middleware, and the systems that make a trained model useful to something that moves in the physical world. That seam is where this programme lives, and it is where his existing work already points.

Intern	Supervisor
Name: Calvin Wang	Name: Iqbal Patel Title: HR & Admin Manager, InGen Dynamics Inc.
Signature: _____	Signature: _____
Date: _____	Date: _____